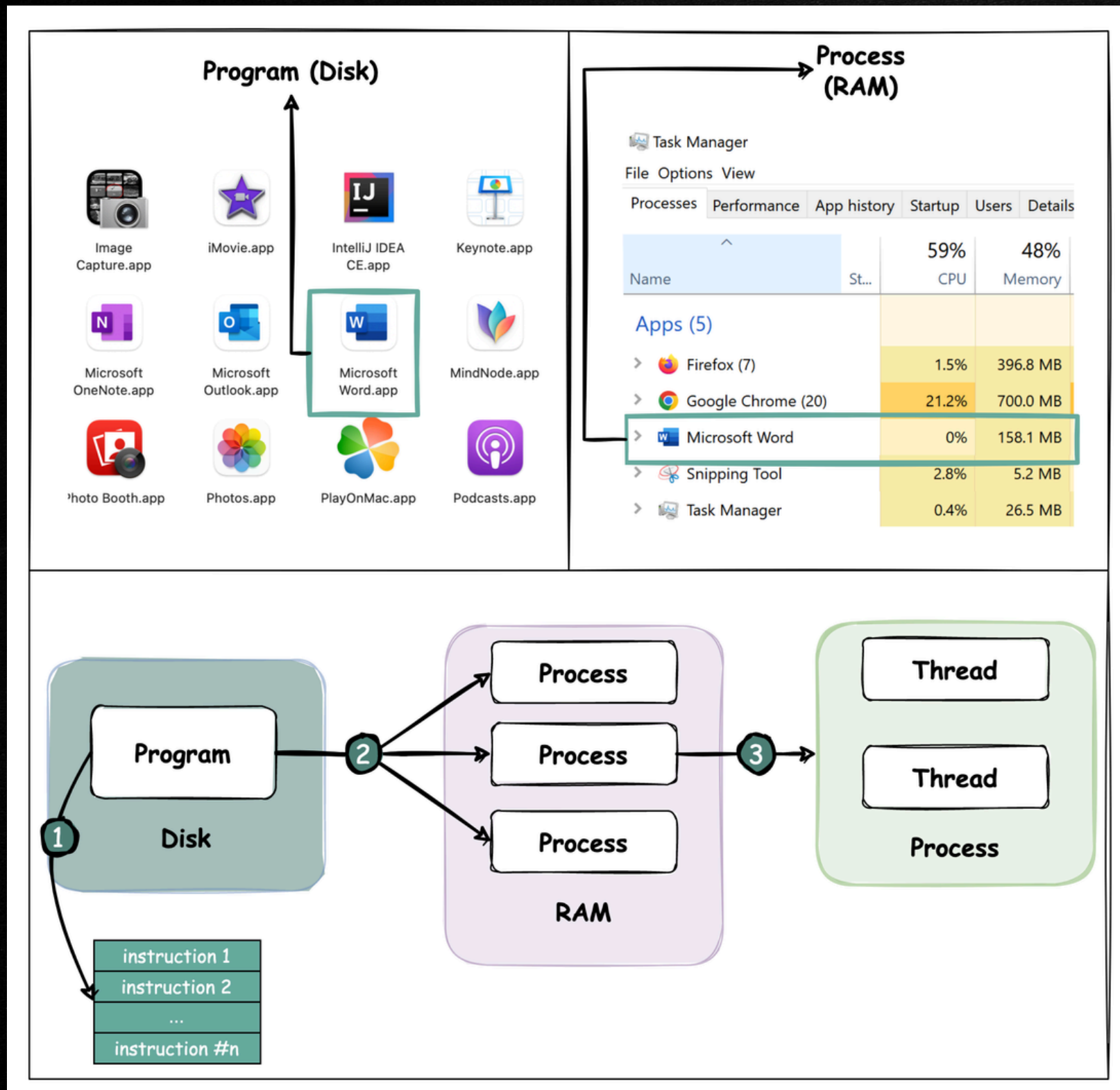
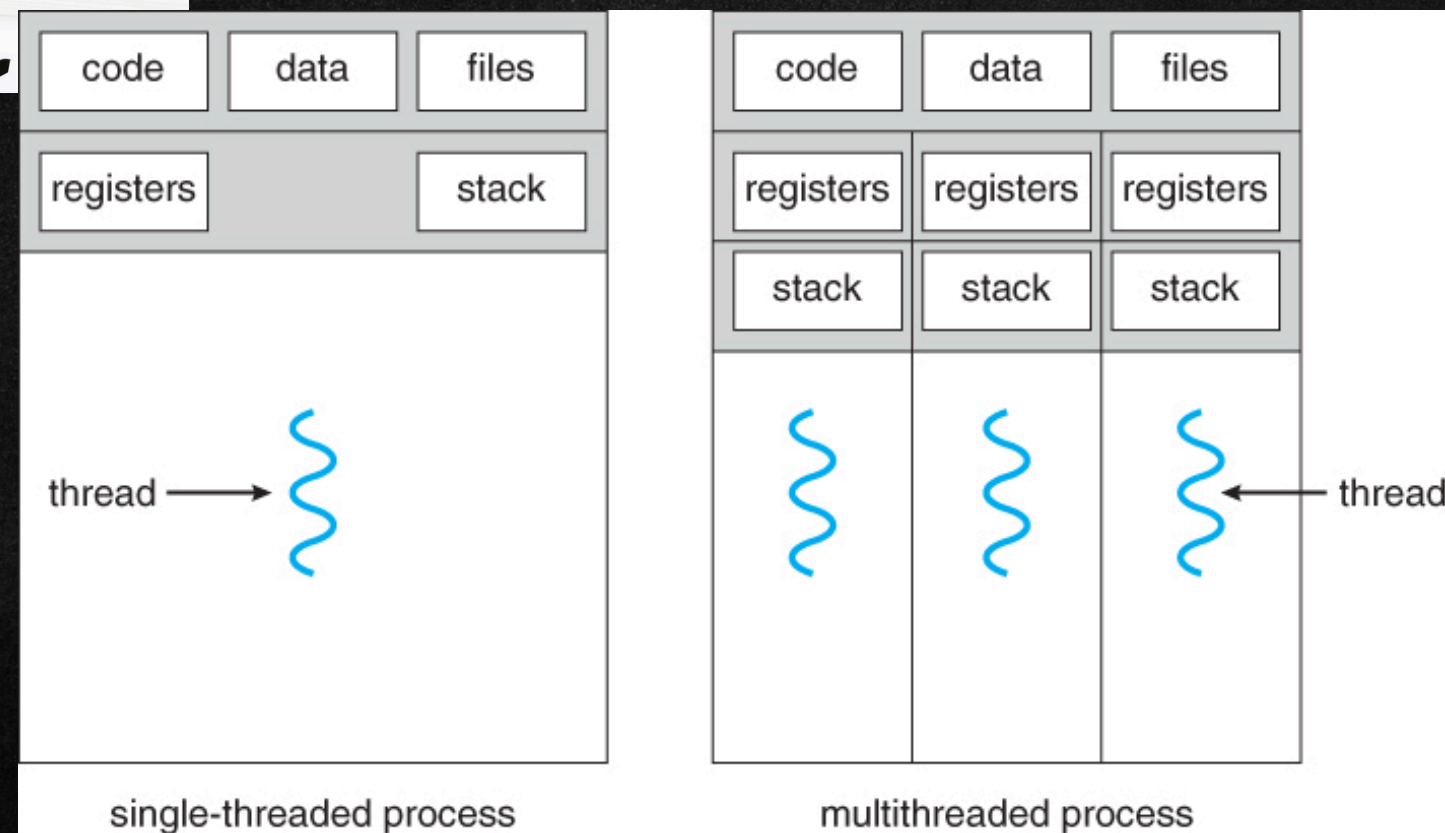


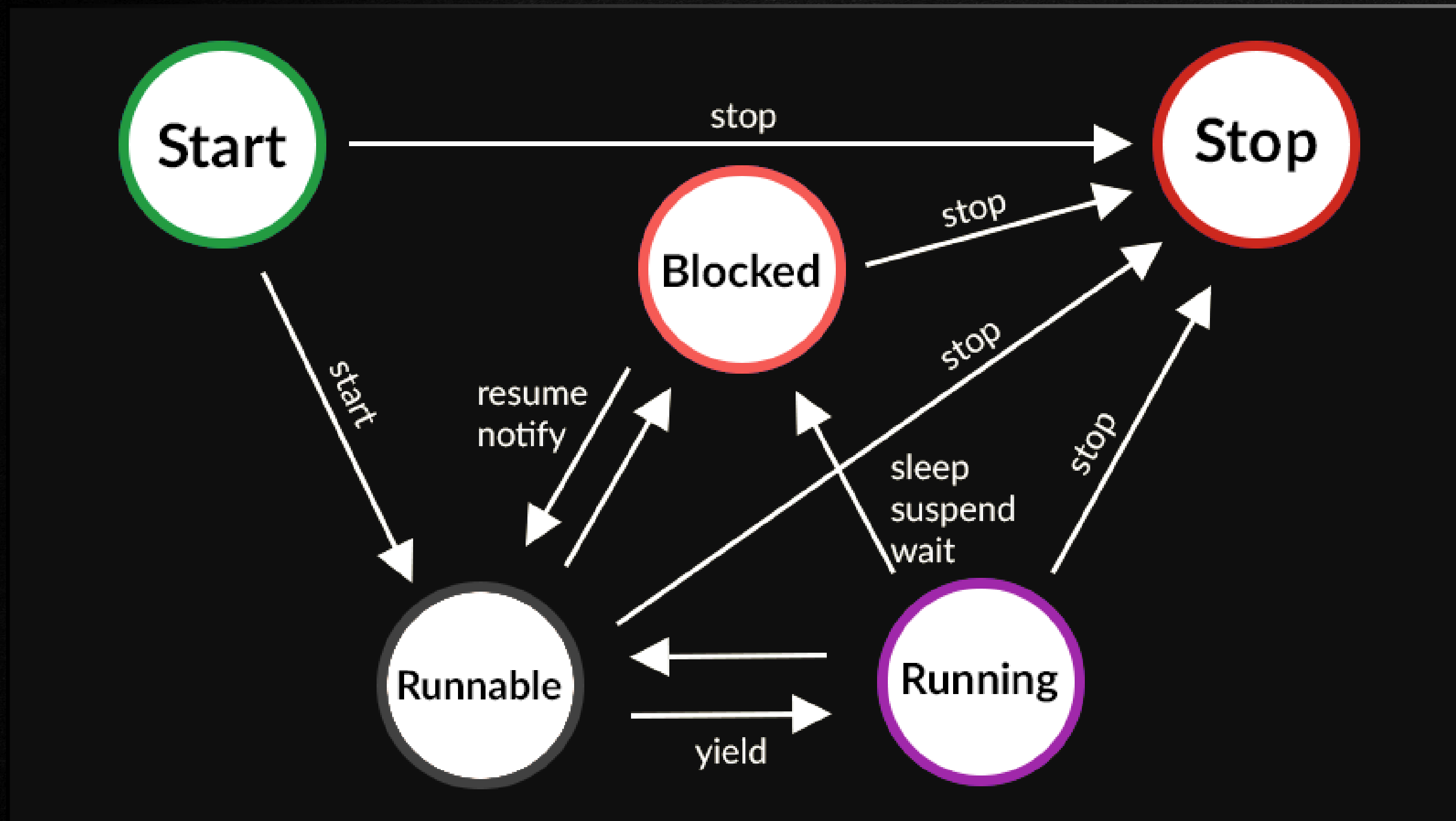
Program vs Process vs Thread



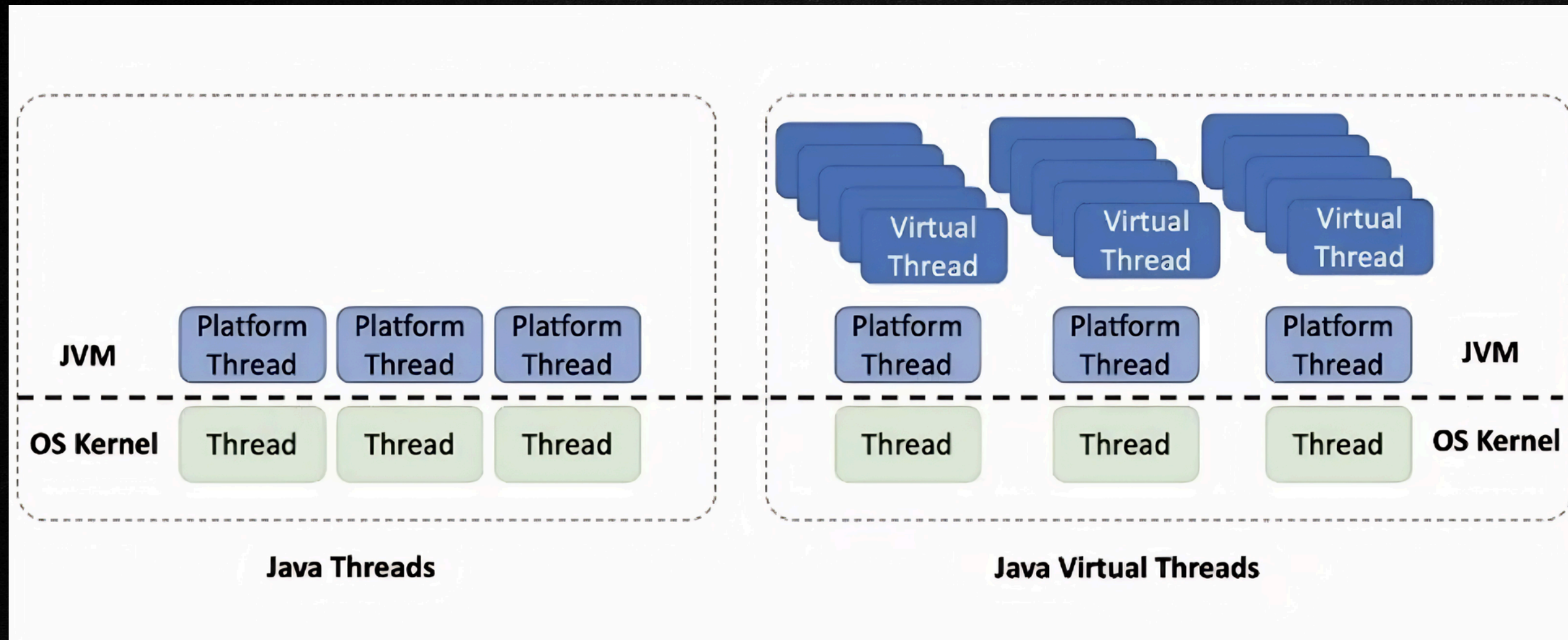
Single vs Multithreaded Processes



Thread States in Java



Java Thread Before and After Java 21



- An 8-core CPU can only ever run 8 threads at one time.
- Your application can have more threads in the RUNNABLE state, but that means they are waiting for actual CPU time from the OS kernel.

Runnable and Callable

Runnable and Callable are functional interfaces used to define a task that can be executed concurrently by a thread.

Both runnable and callable interfaces are designed for classes. Their instances are supposed to be executed by another thread.

- A callable interface throws the checked exception and returns the result.
- A runnable interface, on the other hand, does not return a result and cannot throw a checked exception.

Runnable Interface - Java 1

Runnable — a task with no return value

It is required to create a thread when a runnable interface is implemented through an object. The thread contains a single method named `run()` that does not return any value or accept any parameters.

```
Runnable task = new Runnable() {  
    @Override  
    public void run() {  
        System.out.println("Running on: " + Thread.currentThread().getName());  
    }  
};  
// Submitting to a thread  
Thread thread = new Thread(task);  
thread.start();
```


Callable Interface – Java 5

Callable<V> — a task that returns a result came in java.util.concurrent package in Java 5

- Single abstract method: **V call()** throws Exception.
- Callable Returns a value wrapped in a Future<V> when submitted to an executor
- Use it whenever the task produces a result the caller needs. Typically run via **ExecutorService**.

```
Callable<Integer> sumTask = () -> {  
    int sum = 0;  
    for (int i = 1; i <= 100; i++) sum += i;  
    return sum; // returns 5050  
};
```